МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИ, ЭЛЕКТРОНИКИЙН СУРГУУЛЬ
МЭДЭЭЛЭЛ, КОМПЬЮТЕРИЙН УХААНЫ ТЭНХИМ

Тооцоолол ба өгөгдлийн ухаан

#1 - Python программчлал

(introduction, python basics, control-flow)

Өнөөдрийн хичээл

- Хичээлийн тухай
- Тооцоолол гэж юу вэ
- Python хэлний суурь ойлголт
- Математик операторууд
- Хувьсагч, үндсэн өгөгдлийн төрлүүд
- Нөхцөл шалгах
- Давталт

Хичээлийн тухай

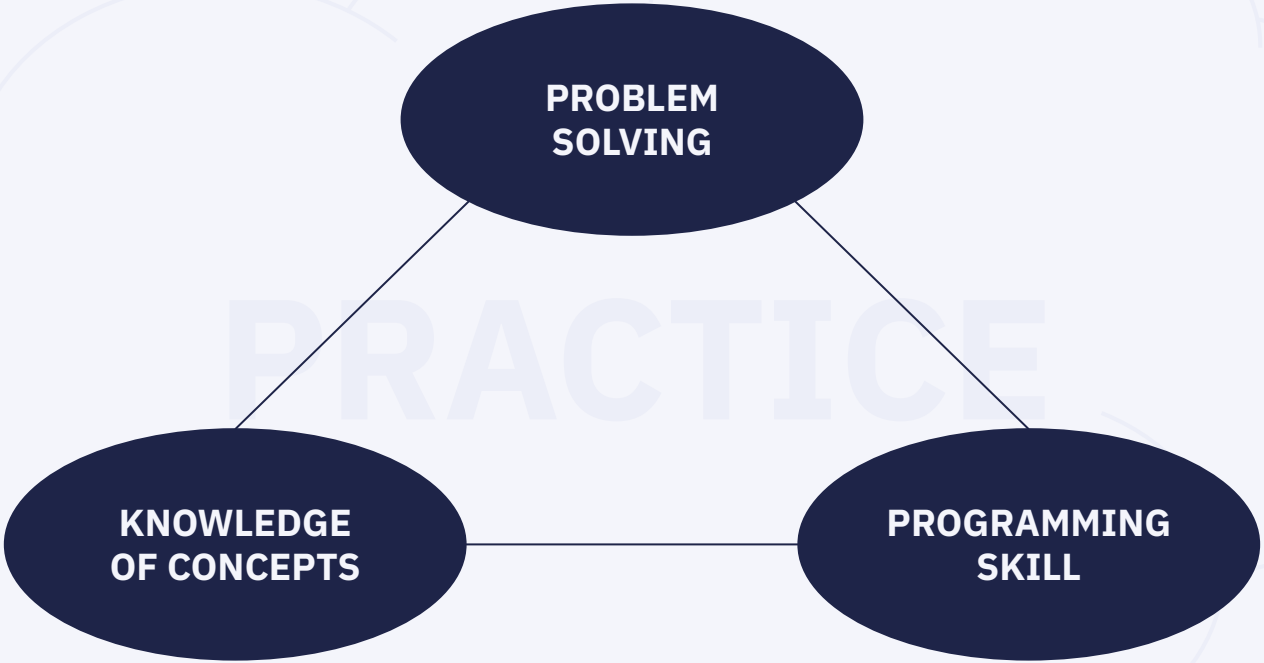
- Агуулга:
  - Python программчлал
    - Өгөгдлийн төрөл, бүтэц, программын урсгал, хийсвэрлэл ...
  - Загварчлал
    - Бодлогын оновчлол, загварчлал
  - Туршилт, симуляци
    - Боловсруулсан загварыг турших, симуляци хийх
  - Машин сургалт
    - Өгөгдөлтэй ажиллах, бүлэглэл, ангиллын алгоритмууд
- Ном:
  - Guttag, John. “**Introduction to Computation and Programming Using Python: With Application to Understanding Data**” 3rd ed. MIT Press, 2021.

1. MIT 6.0001 - Introduction to Computer Science and Programming in Python
2. MIT 6.0002 - Introduction to Computational Thinking and Data Science

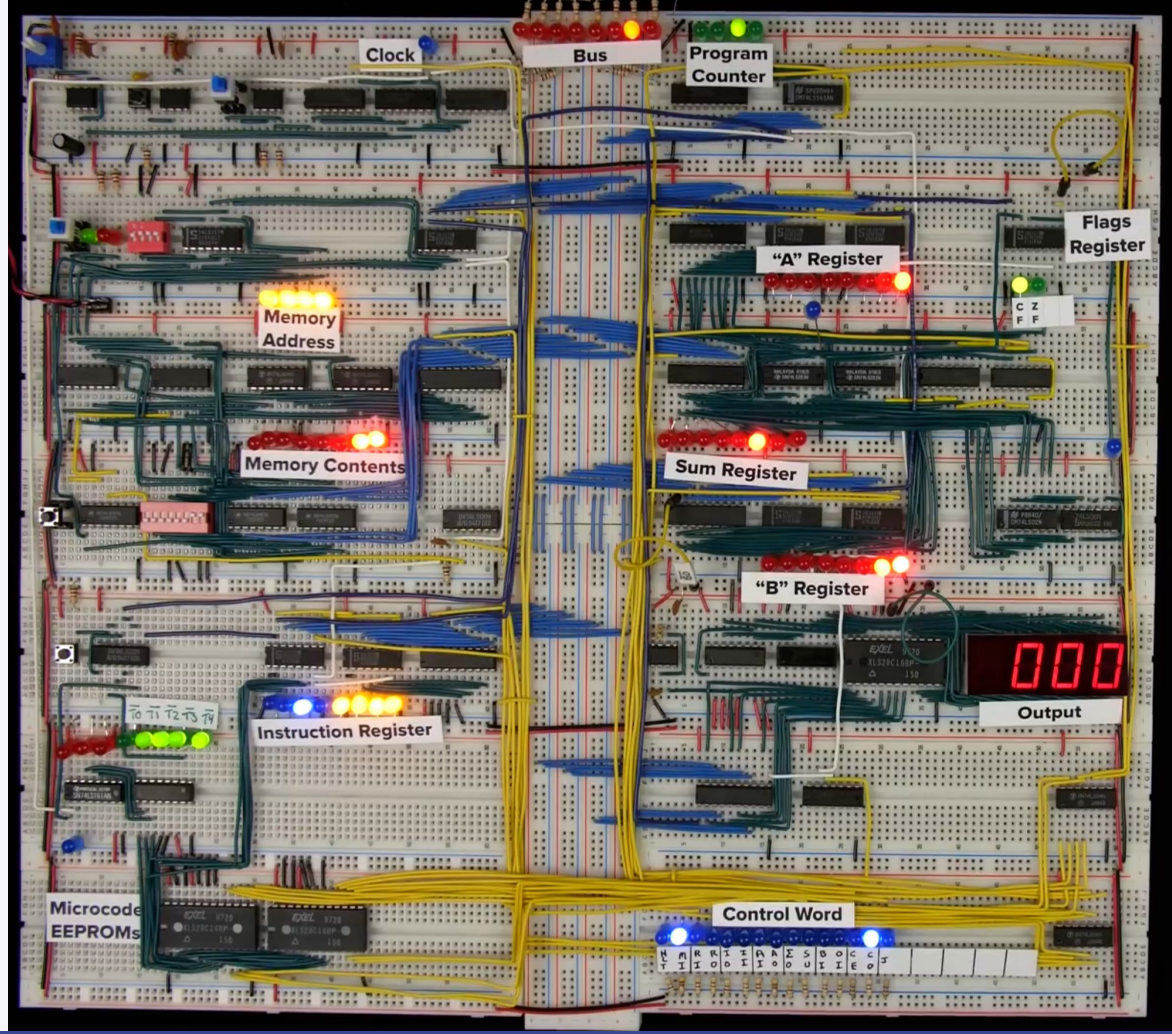
Хичээлийн үнэлгээ

- Лаборатори ~ 48%
- Явц, ирц ~ 22%
- Улирлын шалгалт ~ 30%

Хичээлийн тухай дэлгэрэнгүйг:
https://bilguun0203.github.io/numdocs


PROBLEM SOLVING
KNOWLEDGE OF CONCEPTS
PROGRAMMING SKILL

тооцоолол



[8-bit computer from scratch]

Компьютер юу хийдэг вэ?

Ерөнхийдөө:

- **Тооцоолол** - секундэд хэдэн тэрбум
- **Өгөгдөл хадгалах** - хэдэн зуун/мянган GB

Ямар төрлийн тооцоолол?

- Программчлалын **хэлний өгөгдмөл (built-in)** үйлдлүүд
- Программ зохиогч буюу **та нарын тодорхойлсон (user defined)** үйлдлүүд

** Компьютер зөвхөн та нарын хэлснийг л мэднэ, гүйцэтгэнэ **

Мэдлэгийн төрөл

- **Declarative knowledge** - мэдээллийг шууд илэрхийлэх, **баримт, ойлголт**од тулгуурласан мэдлэг, “юу” гэсэн асуултад хариулагдана.
  - Бууз бол хоол
- **Imperative knowledge** - **арга, зааварчилгаа** агуулсан мэдлэг, “яаж”, “хэрхэн” гэсэн асуултад хариугдана.
  - Бууз хэрхэн хийх вэ?
    1. Гурил зуурна
    2. Мах хэрчиж, бэлтгэнэ
    3. Гурилаа илдэнэ
    4. Чимхэнэ
    5. Жигнэнэ

Жишээ

- “x”-с язгуур авах (√x) гэдэг нь **y*y=x** нөхцлийг хангах “**y**” утгыг олох юм.
- **x**-с язгуур аваад гарах утгыг олох нэг арга нь (x=16)

  1. Ямар нэг тоо **таамаглана**, “g”
  2. **g*g** нь **x**-тэй **хангалттай ойр** байвал **g**-г хариулт гэж үзээд, зогсоно
  3. Үгүй бол **g** болон **x/g**-н дунджаар **шинэ таамаг** дэвшүүлнэ
  4. Шинэ таамгаар дахин эхнээс нь хангалттай ойр болох хүртэл нь үйлдлүүдээ **давтана**

g	g*g	x/g	(g+xg)/2
3	9	16/3	4.17
4.17	17.36	3.837	4.0035
4.0035	16.0277	3.997	4.000002

Зааварчилгаа гэж юу вэ

1. Энгийн **алхмуудын** дараалал
2. Үйлдэл хийгдэх эсэхийг удирдах (**flow of control**)
3. Хэзээ **дуусгах нөхцөл**

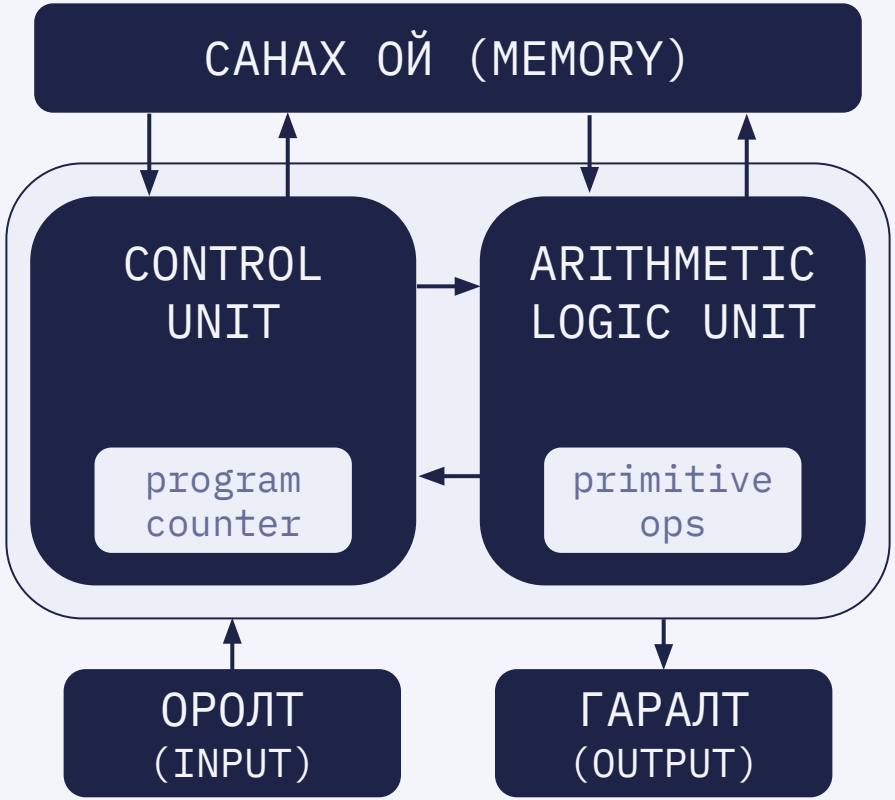
1 + 2 + 3 = **алгоритм**!

Машинд хэрхэн зааварчилгаа өгдөг вэ?

Fixed program компьютер

Stored program компьютер

Компьютерийн ерөнхий архитектур



Central Processing Unit

Stored Program

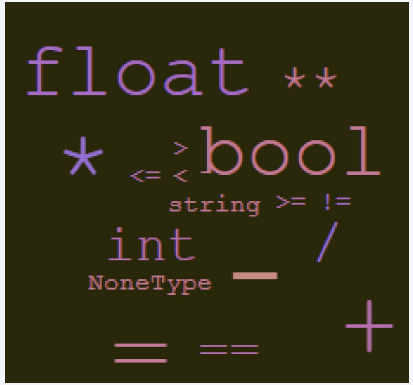
- Компьютерт хадгалагдах программ нь дараах анхдагч зааварчилгаанаас (**primitive instructions**) бүрдсэн байна:
  - Арифметик болон логик
  - Шалгалт (test)
  - Өгөгдөл зөөх
- Тусгай программ (**interpreter**) **зааварчилгааг дарааллын дагуу** нэг, нэгээр нь гүйцэтгэнэ
  - Шалгалтын тусламжтай үйлдлийн дарааллыг удирдана (зааварчилгаа алгасах, дуусгах г.м)
  - Зааварчилгаа дуусмагц зогсоно

Анхдагч үйлдэл

- Алан Тюринг үндсэн 6 үйлдлээр **юуг ч тооцоолох боломжтой**г харуулсан (зүүн, баруун, хэвлэх, унших, арилгах, юу ч хийхгүй)
- Орчин үеийн программчлалын хэлүүд илүү хэрэглэхэд хялбар анхдагч үйлдлүүдтэй болсон
- Үйлдлүүдийг хийсвэрлэх байдлаар **шинэ анхдагч үйлдэл үүсгэж** болно
  - Жишээ нь олон нэмэх үйлдэл үржих үйлдэл болох
- Нэг программчлалын хэлээр тооцоолж болсон зүйлийг өөр программчлалын хэлээр ч тооцоолох боломжтой
- Анхдагч үйлдлүүдийн тусламжтай **илэрхийлэл** (**expression**) гэх цогц үйлдэл үүсгэж болно

Хэл

- **Primitive constructs**
  - Монгол, англи хэл: үг
  - Программчлалын хэл: тоо, тэмдэгт мөр, үндсэн операторууд



Хэлний тухай

- **Syntax - өгүүлбэр зүй**
  - Монгол хэл (өгүүлэгдэхүүн + шууд тусагдхуун + өгүүлэхүүн):
    - “алим жүрж би” → өгүүлбэр зүйн хувьд буруу
    - “би алим идэв” → өгүүлбэр зүйн хувьд зөв
  - Программчлалын хэл (утга + оператор + утга):
    - `“hi”5` → өгүүлбэр зүйн хувьд буруу
    - `3.2 * 5` → өгүүлбэр зүйн хувьд зөв

Хэлний тухай

- **Static semantics - статик утга зүй**
  - Монгол хэл:
    - “алим алхав” → статик утга зүйн хувьд буруу боловч өгүүлбэр зүйн хувьд зөв
  - Программчлалын хэл:
    - `3.2 * 5` → статик утга зүйн хувьд зөв
    - `3 + “hi”` → статик утга зүйн хувьд буруу

Хэлний тухай

- **Semantic** - syntax, static semantics-н хувьд алдаагүй байх
  - Монгол хэлэнд нэг бичвэр олон утга илэрхийлж болно:
    - “энэ манай ангийн хэрэгсэл”
    - “унага идэв”
  - Программчлалын хэл цор ганц утга илэрхийлнэ
    (гэхдээ тэр нь программ зохиогчийн хүссэн үр дүн биш байж болно)

Алдаа хаана гардаг вэ

Syntactic errors

- Элбэг тохиолдоно, хялбар олно

Static semantic errors

- Зарим программчлалын хэл программ ажиллахаас өмнө шалгадаг
- Үйлдлийн тодорхойгүй байдал үүсэх боломжтой

Semantic алдаагүй боловч программистын хүссэн үйлдэл биш

- Программ зогсох, осолдох
- Программ зогсолтгүй ажиллах
- Программ буруу хариулт өгөх

Python программ

- **Программ** гэдэг нь
  - **Тодорхойлолт** (**definition**),
  - **Тушаалын** (**command**) дараалал
- Python interpreter программын
  - Тодорхойлолтыг **тооцоолно** (**evaluate**)
  - Тушаалыг **гүйцэтгэнэ** (**execute**)

- Python-д тушаалыг шууд **shell**-р ажиллуулах эсвэл **файл**д хадгалан shell-р ажиллуулж болно

Объект

- Программ өгөгдлийн объект (**data objects**)-тэй ажиллана
- Объект нь төрөлтэй, төрөл нь юу хийх боломжтойг тодорхойлно
  - Сурагч бол хүн, тиймээс алхана, иднэ, бодно
  - Мод бол ургамал, тиймээс ургана
- Объект нь хоёр төрөл
  - **Scalar** - цааш хуваагдахгүй (анхдагч)
  - **Non-scalar** - хандах боломжтой дотоод бүтэцтэй (анхдагч төрлүүдийн нэгдэл)

Скаляр объектууд

- `int` - бүхэл тоо (**integer**), ж. нь: 5
- `float` - бодит тоо (**real numbers**), ж. нь: 3.14
- `bool` - **Boolean** буюу True болон False
- `NoneType` - нэг утгатай **тусгай** төрөл, None
- `type()` гэж объектын төрлийг харж болно

```
>>> type(5)
int
>>> type(3.0)
float
```

Python shell-д өгөх тушаал

Enter дарсны дараа гарах үр дүн

Төрөл хувиргах (cast)

Объектын төрлийг нэгээс нөгөө рүү хувиргах боломжтой

- `float(3)` бүхэл тоо 3-г бодит тоо 3.0 болгоно
- `int(3.9)` бодит тоо 3.9-г бүхэл тоо 3 болгож, бүхэл хэсгийг авна

```
>>> type(3.9)
float
>>> type(int(3.9))
int
```

Консоль руу хэвлэх

- Программаас хэрэглэгчид ямар нэг зүйл хэвлэж харуулахдаа `print` тушаалыг хэрэглэнэ

```
In [1]: 3+2
Out[1]: 5
```

“Out” гэсэн нь зөвхөн shell-тэй ажиллаж буй хүнд харагдана

```
In [2]: print(3+2)
5
```

“Out” байхгүй гаралт нь хэрэглэгчид харуулахаар хэвлэсэн. Shell ашиглаагүй үед ч харагдана

Илэрхийлэл (Expressions)

- **Объект болон операторуудыг нэгдл**ээр илэрхийлэл үүсгэнэ
- Илэрхийлэл нь **утгатай** бөгөөд мөн өгөгдлийн төрөлтэй
- Илэрхийлэх бичих дүрэм:
  - `<object> <operator> <object>`

Тоон төрлийн операторууд

- `i + j` → **нийлбэр**
- `i - j` → **ялгавар**
- `i * j` → **үржвэр**
- `i / j` → **хуваах**

хоёул int бол хариу нь int байна
аль нэг нь эсвэл хоёул float бол хариу нь float байна

хариу нь float

- `i % j` → i-г j-д хуваагаад гарах **үлдэгдэл**, бүхэл тоо
- `i ** j` → i -г j **зэрэгт дэвшүүлэх**

Үйлдлийн дараалал

- Хаалтын тусламжтай ямар үйлдлийг эхэнд хийх вэ? гэдгийг тодорхойлж болно
- **Операторын эрэмбэ** (хаалтгүй тохиолдолд)
  - `**`
  - `*` `/` `//` `%`
  - `+` `-` зүүнээс баруун тийш

Хувьсагч болон түүний утга

- Тэнцүүгийн тэмдэг нь хувьсагчид **утга олгох** оператор юм

хувьсагч

утга

```
pi = 3.14159
pi_approx = 22/7
```

- Утга компьютерийн санах ойд хадгалагдана
- Утга олгох үйлдэл нь утгад нэр өгч байгаа гэж ойлгож болно
- Хувьсагчид харгалзах утгыг түүний нэрээр нь дуудаж авч болно

Илэрхийллийг хийсвэрлэх

Утгад нэр буюу хоч өгөх:

- Нэг утгыг **нэрээр нь олон дахин ашиглах** боломжтой
- Дараа нь кодонд өөрчлөлт оруулахад хялбар

```
pi = 3.14159
radius = 2.2
area = pi * (radius ** 2)
```

(3.14159) (2.2)

Программчлал vs Математик

- Математикт х-г олдог бол программчлалын хэл утга хадгалдаг

```
pi = 3.14159
radius = 2.2
# area of circle
area = pi * (radius ** 2)
radius = radius + 1
```

утга олгох үйлдэл
* баруун талын илэрхийлэл бодогдож, утга гаргана
* зүүн талд байрлах хувьсагчид утга олгогдоно
* үүнийг мөн `radius += 1` гэж бичиж болно

Хувьсагчийн утга солих

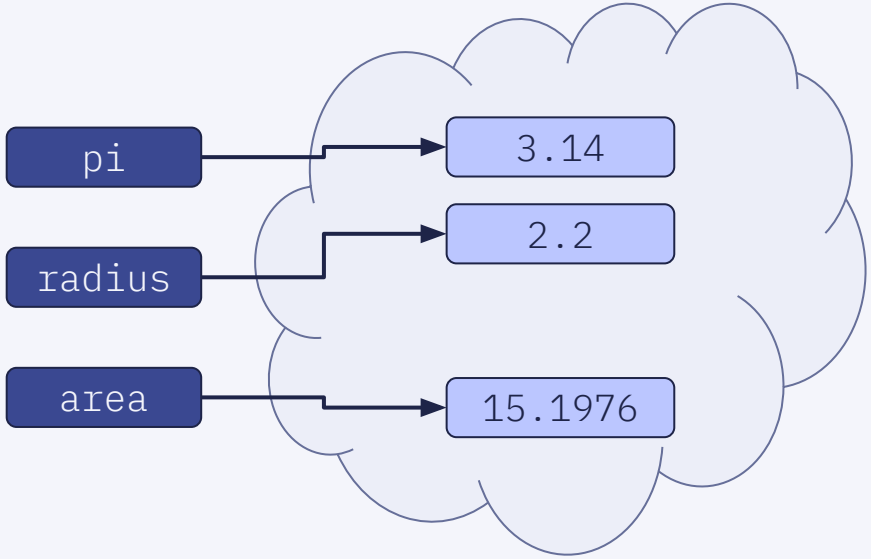
- Хувьсагчийг утга олгох үйлдлийн тусламжтай **өөр утгатай дахин харгалзуулж** болно
- Аль хэдийн тооцоолсон утга дахин тооцоолох хүртэл өөрчлөгдөхгүй

```
pi = 3.14159
radius = 2.2
area = pi * (radius ** 2)
radius = radius + 1
```

Хувьсагчийн утга солих

- Хувьсагчийг утга олгох үйлдлийн тусламжтай **өөр утгатай дахин харгалзуулж** болно
- Аль хэдийн тооцоолсон утга дахин тооцоолох хүртэл өөрчлөгдөхгүй

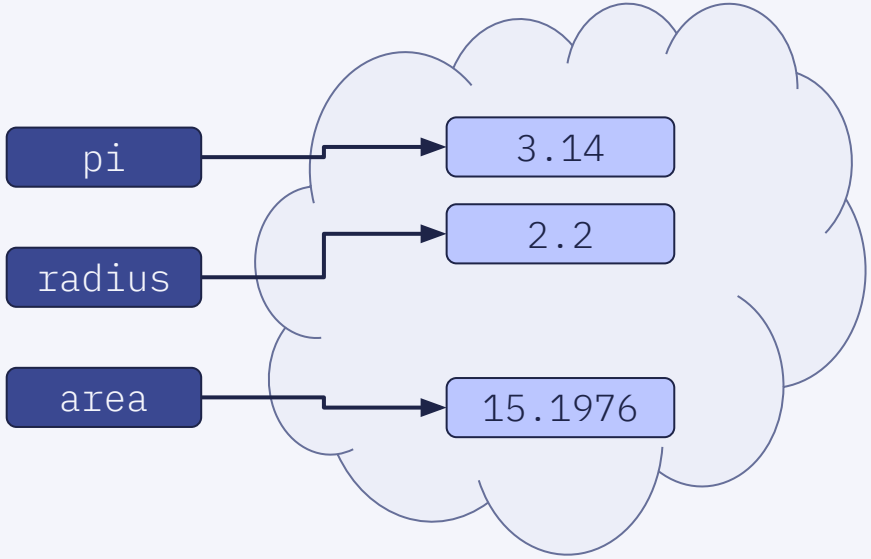
```
pi = 3.14159
radius = 2.2
area = pi * (radius ** 2)
radius = radius + 1
```



Хувьсагчийн утга солих

- Хувьсагчийг утга олгох үйлдлийн тусламжтай **өөр утгатай дахин харгалзуулж** болно
- Аль хэдийн тооцоолсон утга дахин тооцоолох хүртэл өөрчлөгдөхгүй

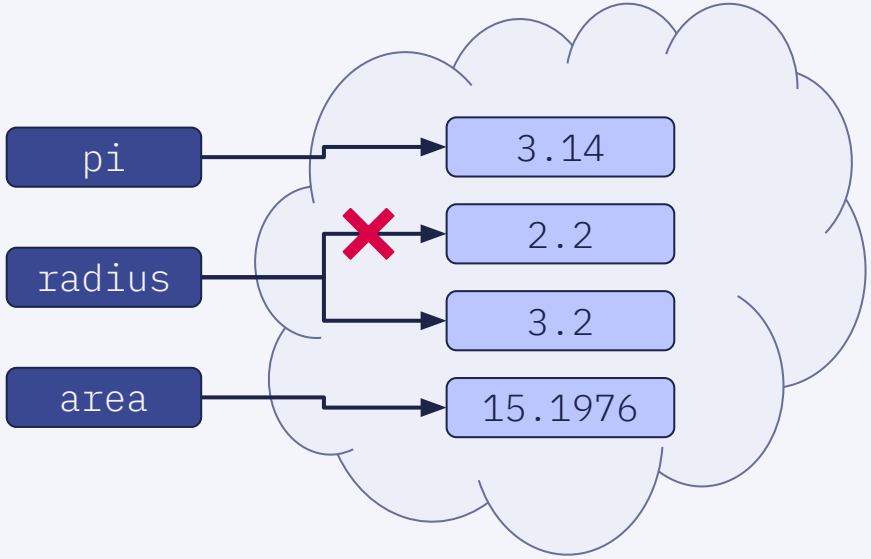
```
pi = 3.14159
radius = 2.2
area = pi * (radius ** 2)
radius = radius + 1
```



Хувьсагчийн утга солих

- Хувьсагчийг утга олгох үйлдлийн тусламжтай **өөр утгатай дахин харгалзуулж** болно
- Аль хэдийн тооцоолсон утга дахин тооцоолох хүртэл өөрчлөгдөхгүй

```
pi = 3.14159
radius = 2.2
area = pi * (radius ** 2)
radius = radius + 1
```



Тэмдэгт мөр (strings)

- Үсэг, тусгай тэмдэгт, хоосон зай, тоо
- **Давхар эсвэл дан хашилт дотор бичнэ**

```
hi = "hello"
```

- Тэмдэгт мөр **залгах** (**concatenate**)

```
name = "bilguun"
greet = hi + name
greeting = hi + " " + name
# concatenation-с гадна төрөл бүрийн үйлдэл байгаа
another_greeting = hi + "  " + name * 3
```

Оролт / Гаралт - print

- Консоль руу гаргах буюу **хэвлэх**
- Дуудах түлхүүр үг: print

```
x = 1
print(x)
x_str = str(x)

print("my fav num is", x, ".", "x = ", x)

print("my fav num is " + x_str + ". " + "x = " + x_str)
```

Оролт / Гаралт - `input`

- Хашилтан доторхыг хэвлэнэ
- Хэрэглэгч ямар нэг юм бичээд Enter дарна
- Орж ирсэн утгыг хувьсагчид онооно

```
text = input("Юу ч хамаагүй бичнэ үү... ")
print(5 * text)
```

- `input` нь **үргэлж тэмдэгт мөр төрлийн утга буцаана**, тоон төрөл болгохын тулд төрөл хувиргах шаардлагатай

```
num = int(input("Тоо бичнэ үү... "))
print(5 * num)
```

Харьцуулах операторууд - `int`, `float`, `string`

- `i` болон `j`-г хувьсагчийн нэр гэж үзье
- Доорх харьцуулалтууд **Boolean** утга буцаана

```
j > j
i >= j
i < j
i <= j
i == j → тэнцүү эсэхийг шалгана, i болон j адилхан бол True буцаана
i != j → тэнцүү биш эсэхийг шалгана, i болон j адил биш бол True буцаана
```

Логик оператор

- a болон b-г Boolean төрлийн хувьсагч гэж үзье

`not a` → a нь `False` бол `True` буцаана,
a нь `True` бол `False` буцаана

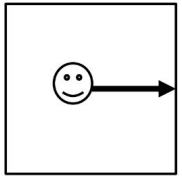
`a and b` → хоёул `True` бол `True` буцаана

`a or b` → аль нэг нь True бол `True` буцаана

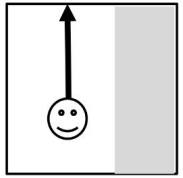
A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Харьцуулалтын жишээ

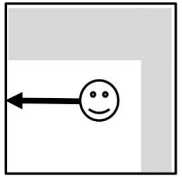
```
current_time = 15
sleep_time = 22
print(sleep_time > pset_time)

has_pen = False
has_paper = True
take_notes = has_pen and has_paper
print(take_notes)
```



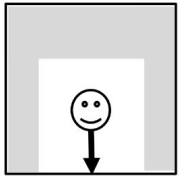
If right clear,
go right



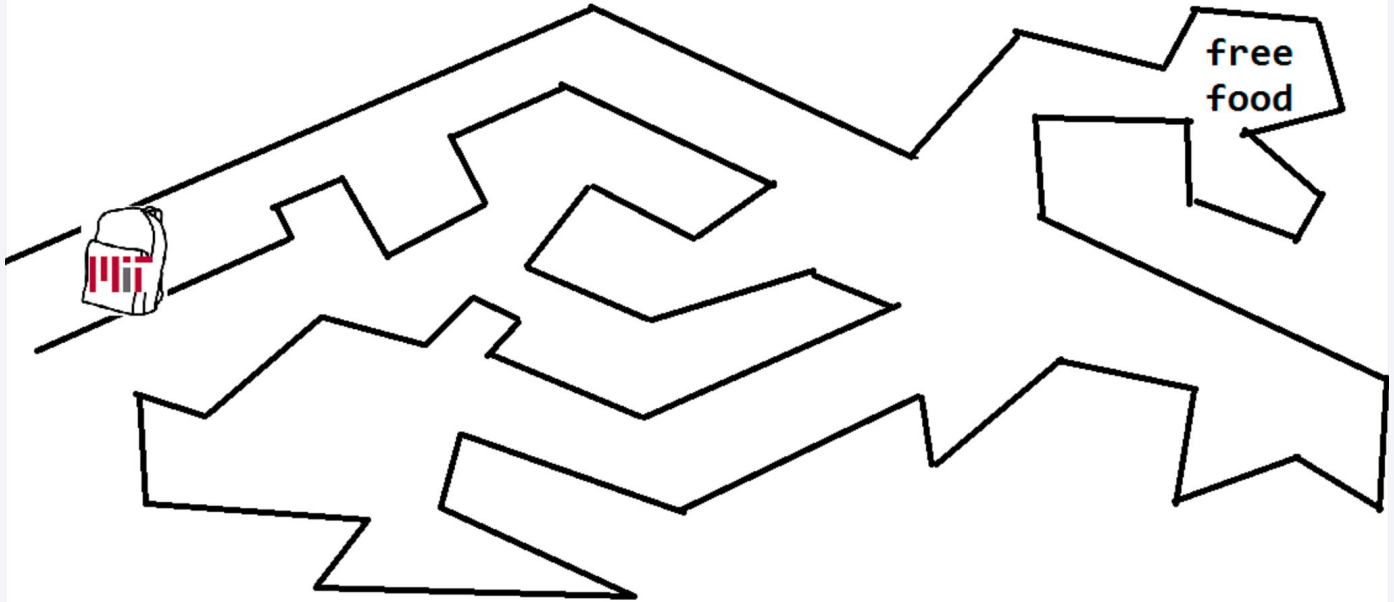
If right blocked,
go forward



If right and
front blocked,
go left



If right , front,
left blocked,
go back



Control Flow - Салбарлах (Branching)

```
if <condition>:
    <expression>
    <expression>
    ...
```

```
if <condition>:
    <expression>
    <expression>
    ...
else:
    <expression>
    <expression>
    ...
```

```
if <condition>:
    <expression>
    <expression>
    ...
elif <condition>:
    <expression>
    <expression>
    ...
else:
    <expression>
    <expression>
    ...
```

- `<condition>` нь `True` эсвэл `False` утгатай байна
- `<condition>` нь `True` бол тухайн блок дахь илэрхийллүүдийг тооцоолно

Indentation

- Python хэлэнд хоосон зайгаар блок кодыг ялгадаг (**indentation**)

```
x = float(input("x тоог оруулна уу: "))
y = float(input("y тоог оруулна уу: "))
if x == y:
    print("x болон y тэнцүү")
    if y != 0:
        print("тиймээс, x/y бол", x / y)
elif x < y:
    print("x нь бага")
else:
    print("y нь бага")
print("төгсөв")
```

= vs ==

```
x = float(input("x тоог оруулна уу: "))
y = float(input("y тоог оруулна уу: "))
if x == y:
    print("x болон y тэнцүү")
    if y != 0:
        print("тиймээс, x/y бол", x / y)
elif x < y:
    print("x нь бага")
else:
    print("y нь бага")
print("төгсөв")
```

`if x = y` бол юу болох вэ?
SyntaxError алдаа заана

Legend of Zelda - Lost Woods

Баруун тийш яваад л байхад газар нь хэзээ ч өөрчлөгдөхгүй

```
if <баруун тийш явсан>:
    <арын фоныг woods_background болгох>
    if <баруун тийш явсан>:
        <арын фоныг woods_background болгох>
        if <баруун тийш явсан>:
            <арын фоныг woods_background болгох>
            гэх мэтчилэн цааш ...
        else:
            <арын фоныг exit_background болгох>
    else:
        <арын фоныг exit_background болгох>
else:
    <арын фоныг exit_background болгох>
```

Legend of Zelda - Lost Woods

Баруун тийш яваад л байхад газар нь хэзээ ч өөрчлөгдөхгүй

```
while <баруун тийш явсан>:
      <арын фоныг woods_background болгох>
<арын фоныг exit_background болгох>
```

Control Flow - while давталт

```
while <condition>:
    <expression>
    <expression>
    ...
```

- `<condition>` нь Boolean утга буцаана
- `<condition>` нь True бол while доторх кодын блокыг ажиллуулна
- Дахин `<condition>`-г шалгана
- `<condition>` нь False болтол дээрх үйлдлүүдийг давтана

while давталтын жишээ

```
You are in the Lost Forest.
************
************
     ☺️
************
************
Go left or right?
```

Программ:

```
n = input("You're in the Lost Forest. Go left or right? ")
while n == "right":
    n = input("You're in the Lost Forest. Go left or right? ")
print("You got out of the Lost Forest!")
```

Control Flow - `while` болон `for` давталт

```
# more complicated with while loop
n = 0
while n < 5:
    print(n)
    n = n+1

# shortcut with for loop
for n in range(5):
    print(n)
```

Control Flow - `for` давталт

```
for <variable> in range(<some_num>):
    <expression>
    <expression>
    ...
```

- Давталт бүр `<variable>` шинэ утга авна
- Хамгийн эхний давталтаар `<variable>` хамгийн бага утгаас эхэлнэ
- Дараагийн давталтаар `<variable>` өмнөх утга +1
- гэх мэт

range(start, stop, step)

- Өгөгдмөл утгууд: `start=0` ба `step=1` (заавал бичих шаардлагагүй)
- `stop-1` утга хүртэл давтана

```
mysum = 0
for i in range(7, 10):
    mysum += i
print(mysum)

mysum = 0
for i in range(5, 11, 2):
    mysum += i
print(mysum)
```

`break` statement

- `break` агуулсан хамгийн дотоод давталт шууд зогсоно
- Тухайн кодын блок доторх үлдсэн илэрхийллүүдийг тооцоолохгүй

```
while <condition_1>:
    while <condition_2>:
        <expression_a>
        break
        <expression_b>
    <expression_c>
```

`break` statement

```
mysum = 0
for i in range(5, 11, 2):
    mysum += i
    if mysum == 5:
        break
        mysum += 1
print(mysum)
```

`for` vs `while` давталт

`for` давталт

- Хэдэн удаа давтахаа **мэднэ**
- `break` ашиглан **эрт зогсоож болно**
- **Тоолуур хэрэглэдэг**
- `for` давталтыг `while` ашиглан **хэрэгжүүлж болно**

`while` давталт

- Давталтын тоонд **хязгаар тавьдаггүй**
- `break` ашиглан **эрт зогсоож болно**
- **Тоолуур тай байж болох боловч** давталт **эхлэхээс өмнө тодорхойлж**, давталт дотор нэмэгдүүлнэ
- `while` давталтыг `for` ашиглан **хэрэгжүүлэх боломжгүй** тохиолдол байна